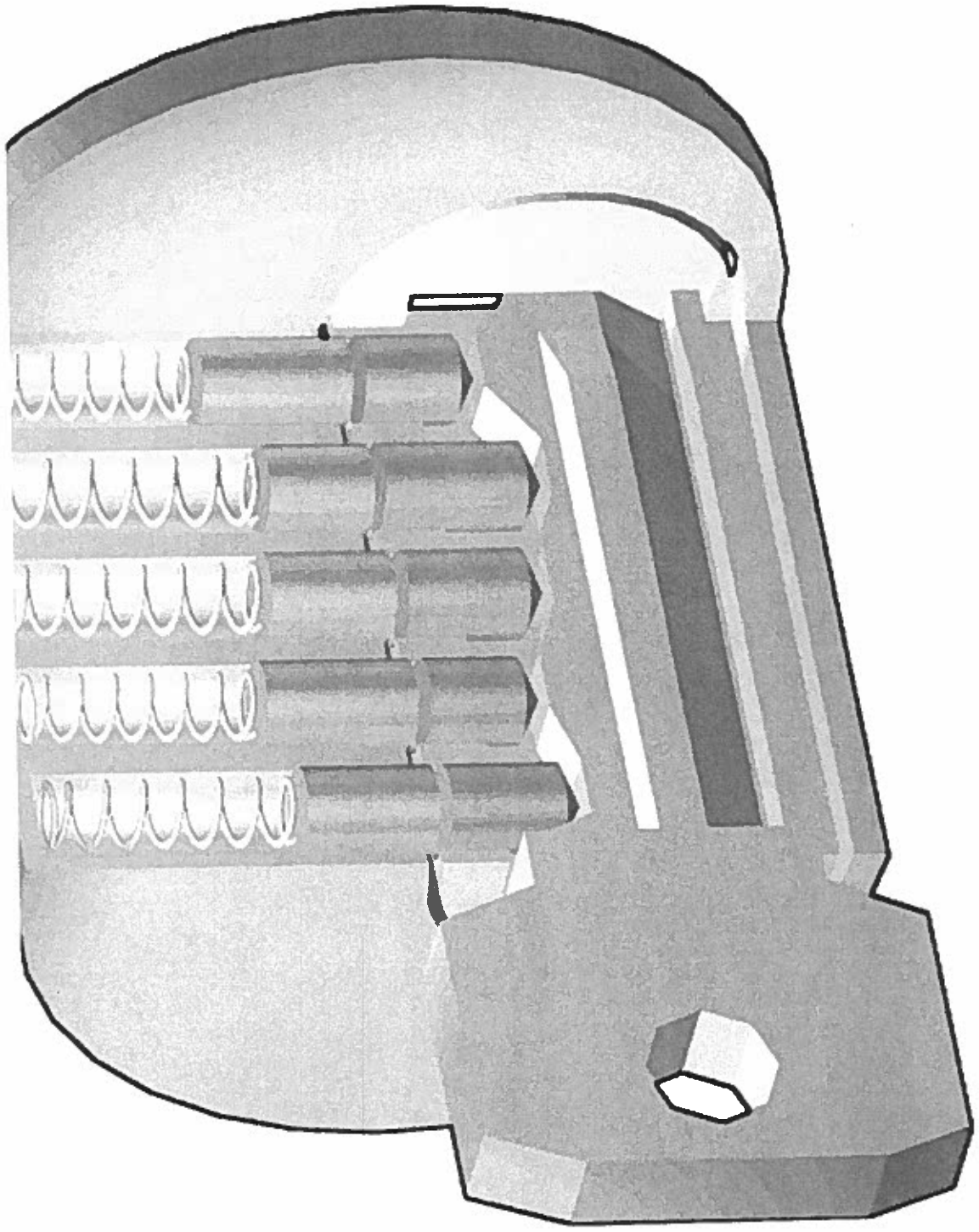
lock

Marcus Alexander
Joshua Satterfield

who is home?

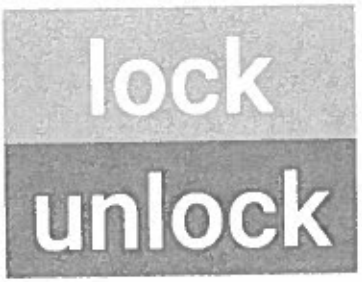


AES-128


CBC

lost
keys?

use key



auth

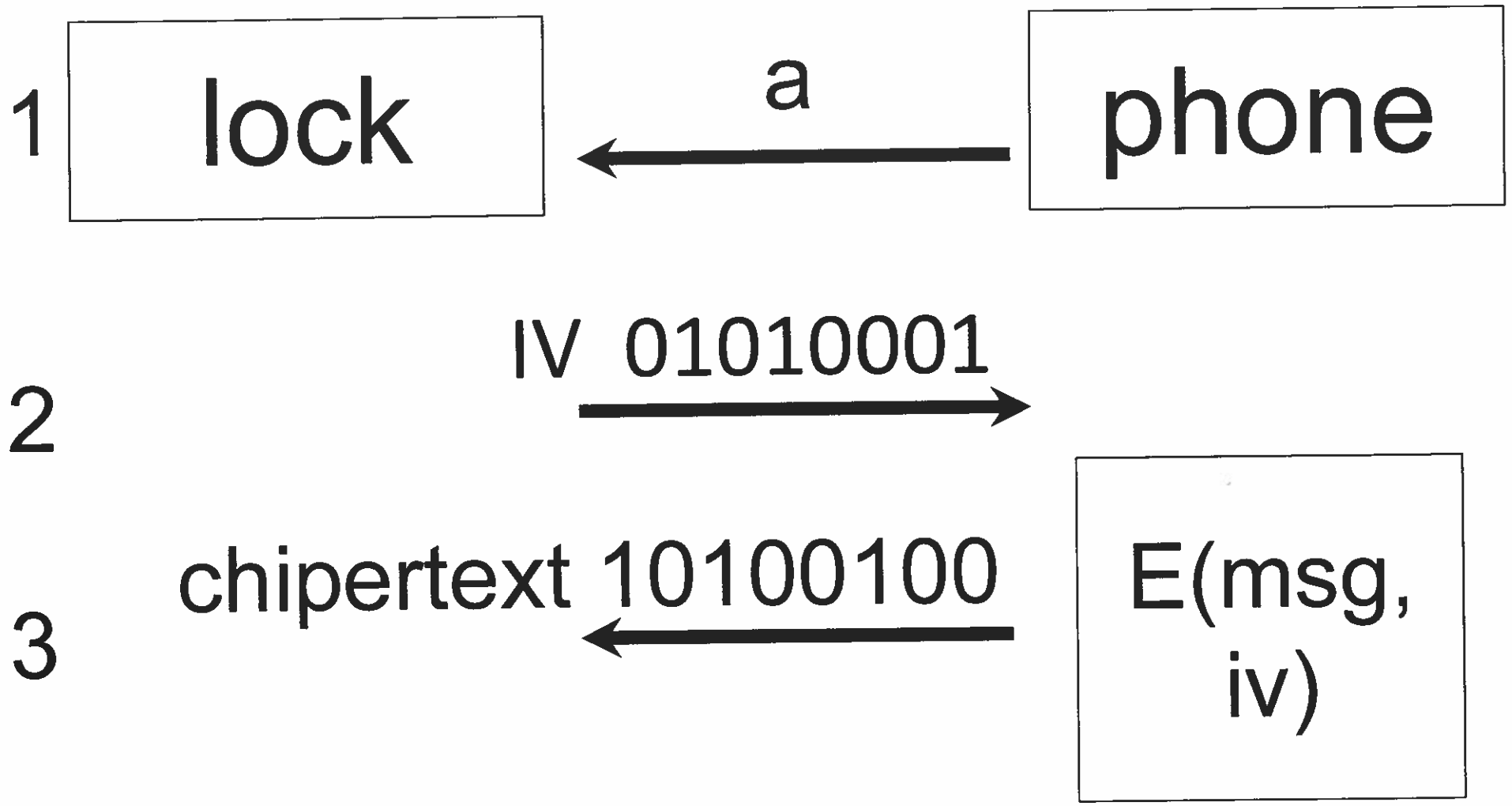
android& arduino.

customiz

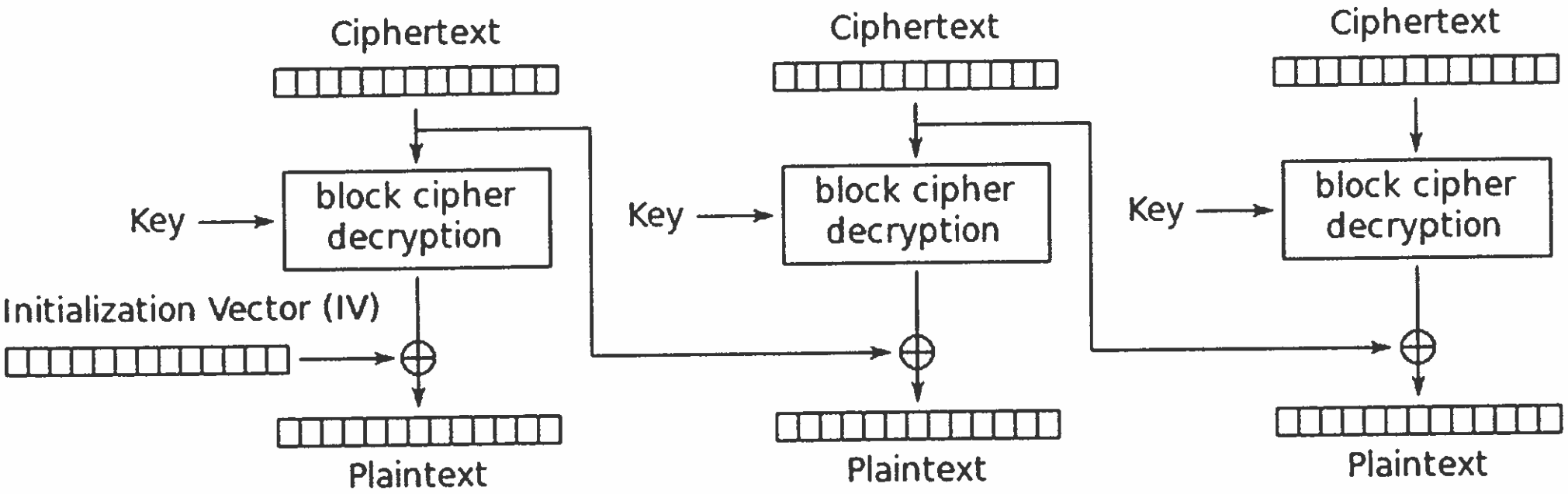
able

simple protocol

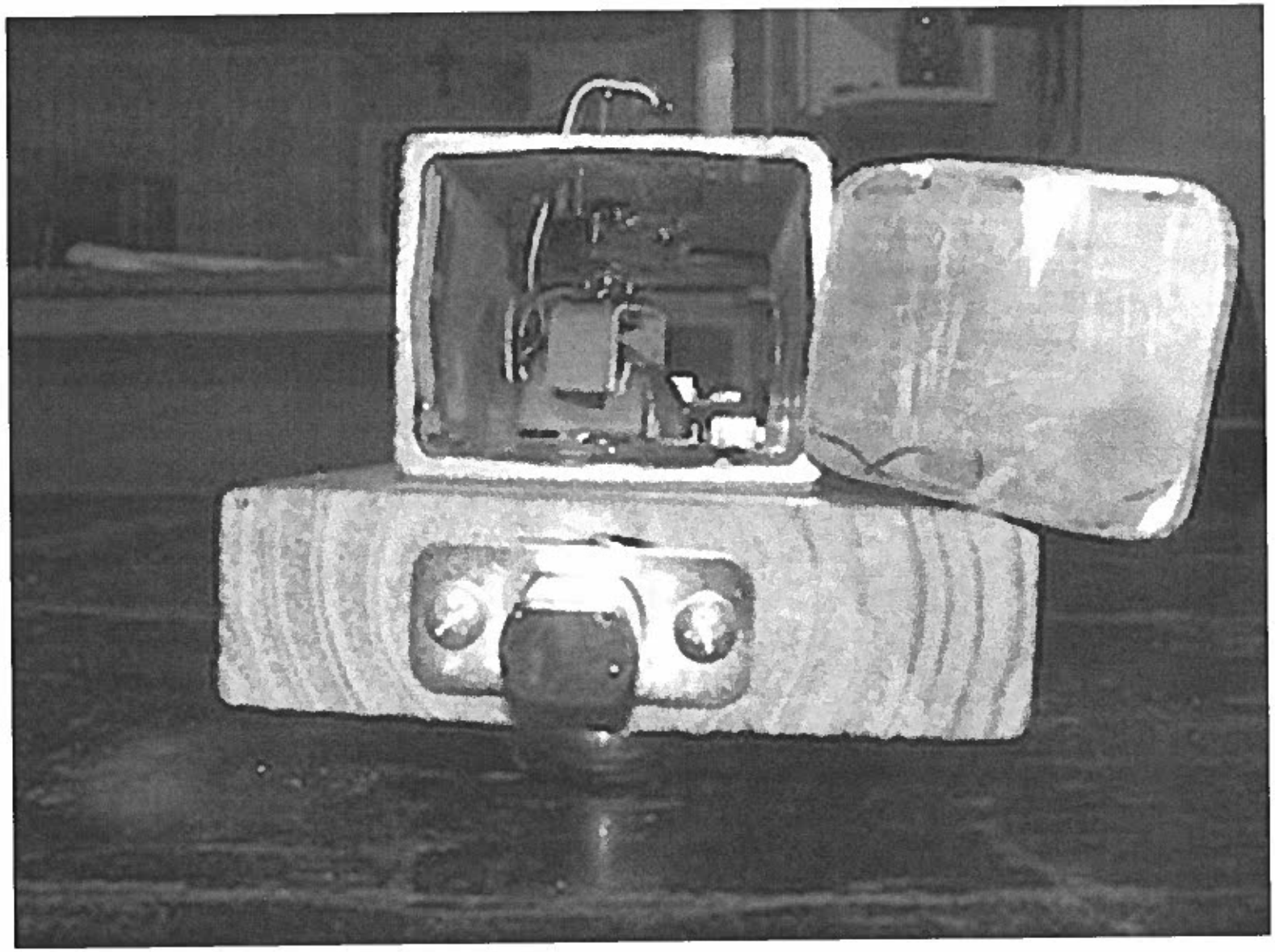
l lock
u unlock
a authorize

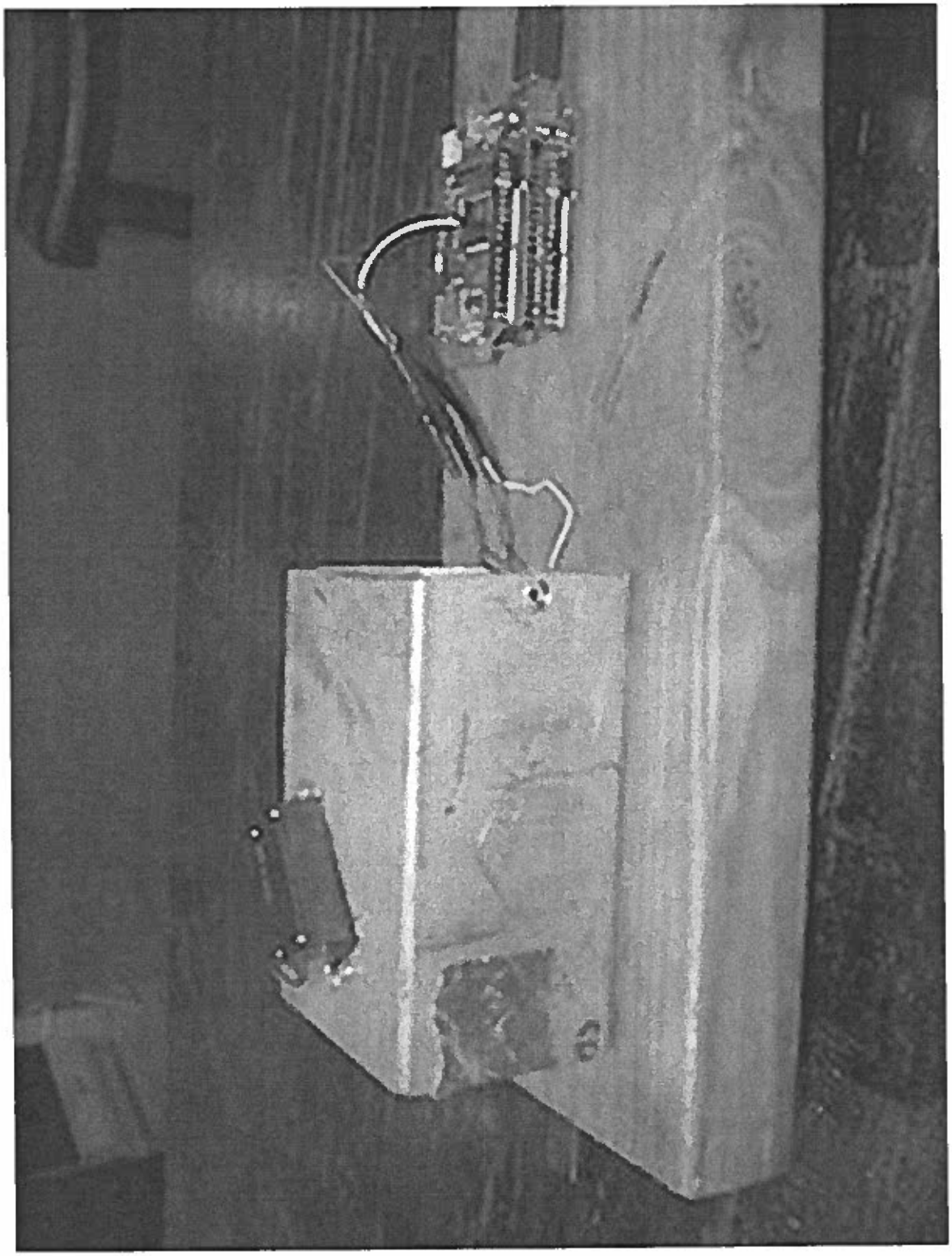

1
lock
a
phone
IV 01010001
2
chipertext 10100100
3
E(msg, iv)

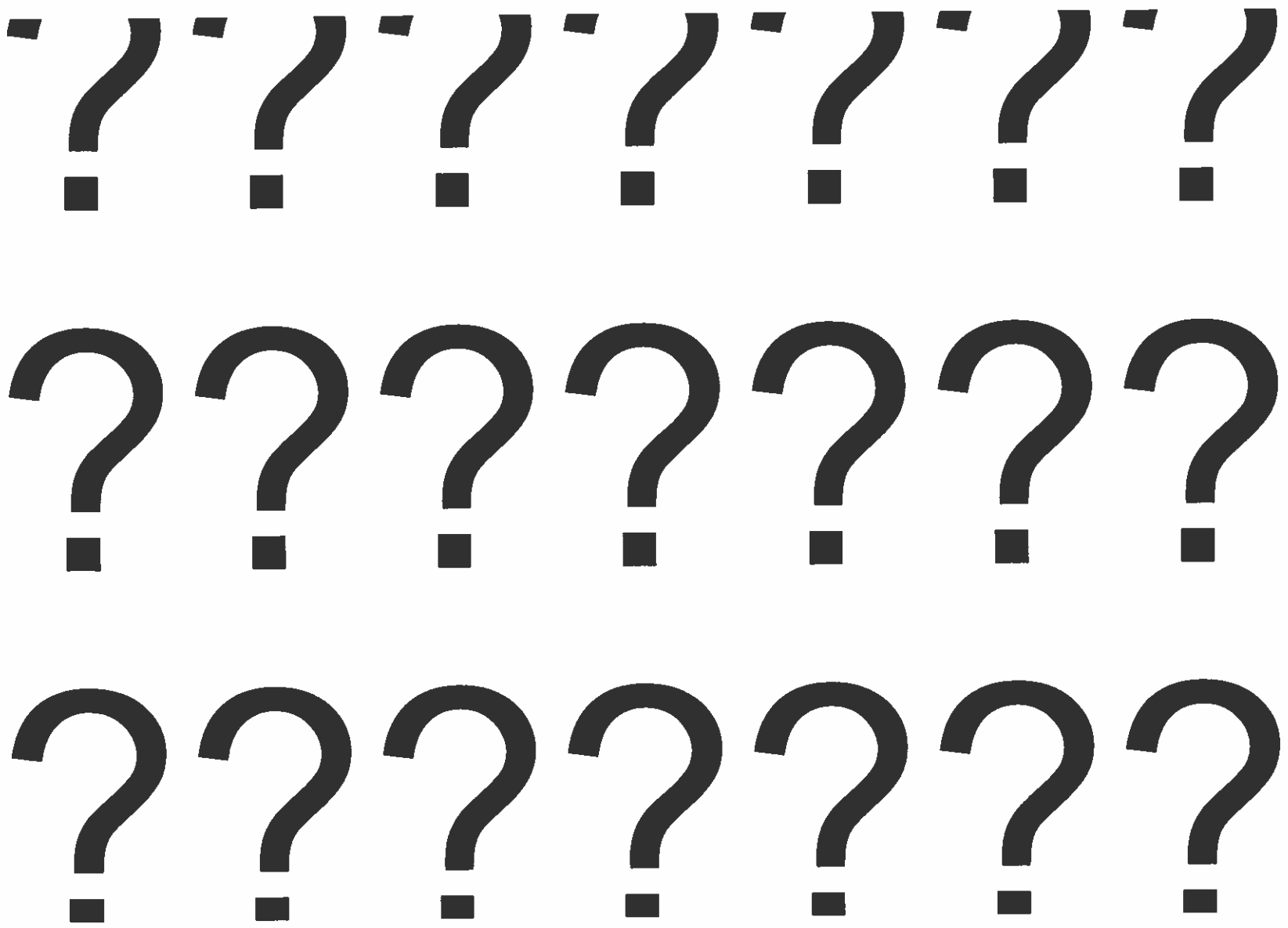
cmd = I
encrypt(msg, iv) =
00100010
decrypt(msg, iv) =
I



Cipher Block Chaining (CBC) mode decryption







Introduction

Joshua N. Satterfield

Advisor: Dr. Yu-Ju Lin

Charleston Southern University

Table of Contents

Abstract

In this project we investigate how to wirelessly lock and unlock a door with off the shelf parts. The goal of this project is to create a system that is both affordable and secure for future homes. We used an Arduino, Arduino Wi-Fi Shield with Android as our platform. We designed a protocol that utilizes the transport layer protocol UDP for communication between an Android smartphone and the platform. We are able to successfully lock and unlock a door wirelessly. There are issues remaining to be resolved. The form factor needs to be reduced for better integration with the user's door. Integrating with the Touch-ID API for added security in case of a lost device must be accomplished as well. This will be a step to avoiding storing the key on the mobile device, increasing safety.

Introduction

Securing homes is an important issue. Unsecured access is a vulnerability for your family. The traditional physical lock system uses a literal lock and key. The lock has pins inside it. The indentation on the key must be the exact counterpart to the length of the pins inside the lock in order for the lock to be turned. When the pins are pushed away the right distance they no longer block the inner mechanism, the tumbler, from turning. This system works, but there are several drawbacks. First, while there are theoretically many combinations of pin lengths, the exact procedure can be circumvented. By hitting the pins with a small tool you can force them all away from the tumbler. Then if you twist it at the correct time you can keep them from falling back in to their protruding states. This can be done in a few seconds by a professional. Second, the number of possible combinations of the pins is limited. The number can vary depending on implementation. A more secure mechanism can be purchased. The high price, however, keeps most home owners away. A more secure modern day solution is required to remedy this situation.

The Internet of Things is a system of devices, primarily embedded, which communicate together for things like information gathering and automation. Kevin Ashton, who coined the term describes a system of devices which operates without human intervention. In his usage the term was originally intended to describe a kind of system, his implementation of which used RFID. The term has now been greatly expanded to refer to other implementations, including the Wi-Fi domain, which allows us to cover a wider range. We will use the same concept to solve the previously mentioned locking systems.

The goal of the project is to research how to implement a modular, customizable system that will be able to secure a future home. We choose a two-part system, integrating with the user's

phone as the key and an Arduino as the lock mechanism. A method of symmetric encryption was chosen for securing communication between the smartphone and the receiver.

In this paper we investigate how to wirelessly lock and unlock a door with off the shelf parts. We first survey the existing systems on the market. We will then discuss the difficulties we encountered in the research and methodology, and how to overcome each obstacle. We then present our experiments and findings in the Implementation section. Finally, we present our conclusion.

Background

In this section we survey the existing related technology. We first investigate US patent 5,942,985. It is a solution which uses a stored key to insure authorization before opening the door. Then we discuss present commercial solutions and their disadvantages. We then present our solution which is able to overcome the shortcomings.

The lock in US patent 5,942,985, from 1999 describes a method of unlocking automatically when the key is near. The lock controller sends a pilot signal, and when the key device detects a valid signal, it sends its key. If the key stored on the lock controller matches the one it receives, it unlocks and notifies the key device. It describes various wireless signals that can be used including radio and microwaves. This system does not allow the door to be controlled without being present. It can also be vulnerable to replay attacks, if it accepts the same unaltered key each time. If so, once someone captures the key, they can open the door with it at any time.

The August Smart Lock replaces the inward facing part of a deadbolt and can be turned manually on the inside. It integrates with the user's smartphone and then uses a mobile application to authenticate the users. It unlocks the door when the phone is near using proximity detection and can lock the door automatically. It allows you to issue keys to

people in your phones contacts. Since these virtual keys are distinct it is more secure. It also allows you to give people access at certain times. It uses Bluetooth and it logs accesses. It has four AA batteries for backup power. However, it has a major flaw.

The password reset system sends your phone a number from one to a million, which is a large problem space for a human but to a computer it can brute force it in a short amount of time. Often websites limit the frequency of attempts but the August software does not contain such precautions [Jmaxxz pt. 2]. As a result, someone can create a program which try all the combinations until one is successful, resetting your password and unlocking your door without really being authorized.

The Kevo lock's uses a Smartphone app for authorization and is unlocked by touching it. It uses Bluetooth but not its security features. Its authentication uses public key encryption and thus is secure [Peter Ha]. It also includes a key fob that is recognized by the lock when the lock is touched. Use uses four AA batteries and does not use home electricity. However, Bluetooth has a limited range, and does not allow it to be controlled remotely.

These systems all have their drawbacks. To overcome these issues, we propose to build a system that is both affordable and secure for future homes. We do not want insecure devices as the gatekeepers of our homes lending themselves to be attack vectors. Instead, a strong 128-bit asymmetric encryption along with proper protocol to further enhance the security. By using a mechanism that unlock your door from the inside only when authorized. We want the "Internet of Things" to integrate into or life without sacrificing security. We also want the convenience of locking and unlocking our doors while we are away without compromising safety.

Goals

We need the communication to be secured for the safety of your own home and family so we will encrypt the communication between devices. We designed a simple protocol for handling the information exchange. We created it without any unnecessary complexity, focusing on the most practical use cases. We need a platform that is small, power efficient and inexpensive. It needs to be small so it can be installed easily and integrate with your homes design.

We have built a compact system which attaches on the face of the deadbolt which allows for installation without removing your door knob. The platform needs to be easy to customize. We have developed two well commented open-source programs which can be installed by simply using the correct USB cable. No extra flashing tools are needed to customize the program and its functionality.

Design and Methodology

We choose the UDP protocol for its simplicity. It is faster and less complex than TCP. Since in the case of dropped data the commands can quickly be sent again by the user we concluded that it is an appropriate tradeoff for speed and simplicity.

To communicate we send short packets with our commands over Wi-Fi. The Arduino then responds with the appropriate action, whether it is to turn the lock to a certain position or respond to authentication attempts.

In order to communicate over Wi-Fi, we first tried using a small wireless component called the XBee ZigBee. It, however, was not designed to communicate using Wi-Fi. Instead it uses its own protocol optimized for mesh networks, with several devices communicating to each other. Therefore, it cannot communicate with a standard smartphone using the Wi-Fi protocol. Its protocol operates differently than Wi-Fi. Although it is wireless, it also runs own a different

incompatible frequency. Likewise, it cannot communicate with the user's home network, or their PC. This prevents the lock from being controlled over the internet without additional hardware.

Thus we attempted utilizing the XBee Wi-Fi module which does use the appropriate frequency and therefore should be able to communicate with the user's home network and smartphone. However, there was no official library for use with the Arduino. We attempted to take advantage of a third party library but we could not initialize the device using that system.

Then we started using the Arduino Wi-Fi shield. The shield comes with a useful network library. Detailed documentation for it is available as well. Using the library documentation, we expanded a simple communication example until it fulfilled our purposes. We added logic after the networking code to react in different ways when certain inputs were received.

In this study, we investigate a scenario of a home environment which needs protection from unwanted entrance. We used Arduino, its Wi-Fi shield, and an Android smartphone to accomplish this research.

Wi-Fi

By using Wi-Fi, we can communicate with the user's cell phone easily. Wi-Fi is more widely available for home networks than Bluetooth. Although phones are typically equipped with Bluetooth, it could not be controlled over the Internet, unlike the Wi-Fi solution. Cellular modules have the advantage that the do not need to be set up with the user's home network information but they have the extra cost of a cellular bill and cannot integrate with their home network directly. So by using W-Fi we achieve lower cost than cellular/GSM and more interoperability than Bluetooth and ZigBee, leave it open to the possibility of control from the user's home network. This allows a system to be constructed that sends commands to the user's house from the internet using the existing receiver on the Arduino. In contrast to Bluetooth or

GSM where an additional receiver would be needed to access Wi-Fi. The disadvantage is that the Arduino needs authorization to use the home network although it can be set up to scan SSIDs. The advantage is that you can communicate as desired from other systems on your network directly or externally using port forwarding.

Protocol

The protocol uses ASCII encoding for its commands, which are a single character. In version one of the protocol the Arduino would enact commands received immediately.

l	u	a
Lock	Unlock	Authorize

In version one of the protocol the Arduino would enact commands received immediately. In version two first the phone must send the authorization command, then wait for a response. The Arduino responds with the AES initialization vector. The phone then encrypts its command using the IV and sends the packet to the Arduino.

Encryption

Challenges

For encryption I began researching the DES protocol. It is now vulnerable to feasible attacks and 3DES is questionable. Despite iterating over the algorithm more times it can still be vulnerable to attacks. It can be slow for software implementations since it was designed for hardware [Pornin]. We then investigated AES – The Advanced Encryption Standard. It has acceptable security for the foreseeable future. An attack has been discovered which allows decryption with less than brute force work but it still has high computational complexity [Bogdanov]. This means it will still take a long time to find the key. I was able to find an Arduino that implemented the basic features of AES and CBC mode. We discuss the functionality of AES and CBC below.

Implementation

We implement a secure method of communication using AES. AES uses an algorithm to encrypt our data. In order to decrypt it you must provide the same key as the one provided to encrypt it, or else the resulting output will be garbage. The key length is typically 128-bits, which is secure enough. This is because because we will not be alive when the computer finishes cracking it by brute force (c.f. Arora). The encrypted packet will naturally be the length of the AES key, which is 128 bits. In the case of a message being longer than a block (the key length) an algorithm must be used to encrypt each block without becoming insecure. We choose the CBC block chaining algorithm because it uses a random IV to prevent replay attacks and it is more secure than EBC. EBC can leave noticeable patterns in the data outputted. The initialization vector is distinct from the key; it is not secret. Thus the IV can be sent over the wire.

The key is implemented as a byte array. When using hex literals to represent bytes compactly it looks like

```
byte[] key = { 0xde, 0xed, 0xbe 0xef, … };
```

In Java bytes are signed. That means you can store positive or negative numbers, unlike unsigned numbers. Since Java excepts unsigned bytes it will not store high numbers in bytes for fear that the upper part of the number will cause them to be interpreted as negative numbers. This we resolved by casting each number to the `int` type. This still fits in a byte, it would only need more space to keep such a number from being interpreted as negative. This is because while unsigned bytes take value from 0 to 225, signed bytes take values from -128 to 127. The leftmost number indicates negation in signed numbers, and takes up on bit. We have enough bits, they just won't be interpreted positive if printed. Here is the way to cast to integers:

```
byte[] key = { (int) 0xde, (int) 0xed, (int) 0xbe (int) 0xef, … };
```

Initialization Vector

We use an initialization vector to increase security. As the initial state for the AES algorithm the IV effects the end result end a manner different than the key which is used. If you encrypt the same message with the same key twice you will get the same result. Thus if someone records you sending an encrypted command signifying "unlock" they could replay this command at any time to manipulate your device. This is a vulnerability that can exist in encrypted systems. To prevent this a random IV is generated each time the module sends an authorization command.

The 16 byte IV comes from the Arduino to the Smartphone and is dealt with using the javax.crypto.Cipher library. The known key (which may be stored on the phone) is combined with the random IV using the AES algorithm and the message is sent back. The next authorization command from a phone will send back a different IV, so the same message will no longer make sense to the Arduino when decrypted using an old IV. This is because it is using an old IV.

Implementation

The implementation uses Android API 23 with java.net.DatagramSocket for the UDP socket on the smartphone. We used an asynchronous function for networking code to keep the UI from blocking. if we just called a long running function the graphics would freeze until the function was running. Since networking can take time this is strongly recommended. We also used the Arduino Wi-Fi library to receive information from the phone. When it receiving a command if it regards the lock it will turn the servo. The servo is wired to this Arduino this way:

Red	Black	White
5V	GND	Pin 9

The lock command turns it to 100 degrees and the unlock command turn it to zero degrees. Choosing 100 degrees instead of 180 keeps to motor from trying to go to far and stuttering.

Using AES CBC with a random IV we prevent the replay attack exploit vector. Using the Arduino microcontroller prototyping platform and Android Studio we built a system which can run on inexpensive and easily obtainable hardware.

One use case our platform does not currently handle is automatically open when you approach the door. Safeties would have to be put in place you solely want to look outside the door. This could be controlled from the mobile app. Handling the approach could be possible by sending out a pilot signal informing the phone that there is a lock nearby, then the phone can reply with a command. The phone would need a program running in the background to react to this situation.

Conclusion

This has been an insightful project. We have learned a lot about practical application of computing concepts in networking, automation and security. We found that using Wi-Fi communication provides an adequate layer for our networking to be built upon and that the speed of transmission was swift. By using the Arduino modular prototyping platform for the receiver we were able to create a quick prototype for that functionality and interface it with the Android application. The platforms we used enabled quick prototyping and the addition of strong encryption for the benefit of the consumer.

Future work

To decrease the size of the device we will investigate the possibility of designing a single chip with both a microcontroller and Wi-Fi. The Arduino uses an AVR microcontroller which can

be obtained at a less expensive price as a standalone part, especially in bulk. We can then create an inexpensive design and pass on the benefits to the consumer.

References

Apple Support. "Apple Computers and Displays: Powering peripherals through USB" https://support.apple.com/en-us/HT204377 Retrieved 2015-12-04

Thomas Pornin. Answer to "Why is AES more secure than DES?". http://stackoverflow.com/questions/3929325/why-is-aes-more-secure-than-des Retrieved 2015-12-04.

Morris Dworkin. "Recommendation for Block Cipher Modes of Operation". NIST. http://csrc.nist.gov/publications/nistpubs/800-38a/sp800-38a.pdf Retrieved 2015-12.

Matthew Green. "How (not) to use symmetric encryption". http://blog.cryptographyengineering.com/2011/11/how-not-to-use-symmetric-encryption.html Retrieved 2015-12.

Chin, Seo-Young. Automatic Locking/unlocking Device and Method Using Wireless Communication. Samsung Electronics Co., Ltd.2, assignee. Patent 5,942,985. 24 Aug. 1999. Print.

"The August Smart Lock." August Smart Lock. Web. Retrieved 25 Feb. 2015. http://august.com/

"UniKey The Evolution of the Key Is Here - As Seen on Shark Tank." UniKey. UniKey. Web. Retrieved 25 Feb. 2015. <http://www.unikey.com/>.

Drf1090 regarding "August Smart Lock." Engadget. 10 Dec. 2014. Web. 26 Feb. 2015. <http://www.engadget.com/products/august/smart-lock/reviews/14f6/>.

Jmaxx. "The August Smart Lock's not so 2-Factor Authentication (Part 1)". <https://jmaxxz.com/blog/?p=476> retrieved 2015-02-01

Jmaxx. "The August Smart Lock's not so 2-Factor Authentication (Part 2)". <https://jmaxxz.com/blog/?p=498> retrieved 2015-02-01

Ha, Peter. "Are Smart Locks Secure, or Just Dumb?" Gizmodo. <http://gizmodo.com/are-smart-locks-secure-or-just-dumb-511093690> retrieved 2015-02-01

Bogdanov, Andrew et. al. "Biclique Cryptanalysis of the Full AES". Microsoft Research. 2011. <http://research.microsoft.com/en-us/projects/cryptanalysis/aesbc.pdf> retrieved 2015-02-01

Arora, Mohit. "How secure is AES against brute force attacks?" EE Times. <http://www.eetimes.com/document.asp?doc_id=1279619> retrieved 2015-02-01

```
/* key: unlock door over udp
 Joshua Satterfield and Marcus
 code based on https://code.google.com/p/boxeeremote/wiki/AndroidUDP
 */
package com.cubesolving.jns.key;

import android.content.Context;
import android.net.DhcpInfo;
import android.net.Uri;
import android.net.wifi.WifiManager;
import android.os.AsyncTask;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.Menu;
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;

import com.google.android.gms.appindexing.Action;
import com.google.android.gms.appindexing.AppIndex;
import com.google.android.gms.common.api.GoogleApiClient;

import java.io.IOException;
import java.net.DatagramPacket;
import java.net.DatagramSocket;
import java.net.InetAddress;
import java.security.InvalidAlgorithmParameterException;
import java.security.InvalidKeyException;
import java.security.NoSuchAlgorithmException;

import javax.crypto.BadPaddingException;
import javax.crypto.Cipher;
import javax.crypto.IllegalBlockSizeException;
import javax.crypto.NoSuchPaddingException;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

import static javax.crypto.Cipher.DECRYPT_MODE;

public class key extends AppCompatActivity {
    /* IV -- Initialization Vector. Should be random and unique. Used as the
    starting state of the AES algorithm. Not Secret. Should receive it from
    the Arduino app.
     */
    byte iv[] = {0x00, 0x00, 0x00, 0x00, 0x00,
            0x00, 0x00, 0x00, 0x00, 0x00,
            0x00, 0x00, 0x00, 0x00, 0x00, 0x01};
    /* AES key. Secret. Should be same as key in Arduino program and it
    should be random. Java uses unsigned bytes so I can't store high literals
    like 0xa7. Now it works if you cast to byte, if interpreted in java it
    could end up negative -- but we don't care, all the bits are there whether
     you interpret the leftmost bit as a sign bit or as a number
     */
    byte key[] = {
            (byte) 0xa7, (byte) 0x74, (byte) 0xde, (byte) 0x71,
            (byte) 0xad, (byte) 0x17, (byte) 0x1d, (byte) 0xbd,
            (byte) 0xed, (byte) 0x3d, (byte) 0x4c, (byte) 0xc9,
            (byte) 0x55, (byte) 0xea, (byte) 0xb6, (byte) 0xb9,
```

```
            (byte) 0x1c, (byte) 0x0b, (byte) 0xed, (byte) 0x76,
            (byte) 0x63, (byte) 0x2d, (byte) 0x69, (byte) 0x42,
            (byte) 0xe7, (byte) 0xd5, (byte) 0x00, (byte) 0xd5,
            (byte) 0x58, (byte) 0x19, (byte) 0xbe, (byte) 0xb4,
//            0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00,
//            0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00, 0x00,
    };

    /* TODO: use as receiving array */
    byte plain[] = {
            // 0xf3, 0x44, 0x81, 0xec, 0x3c, 0xc6, 0x27, 0xba, 0xcd, 0x5d, 0xc3,
0xfb, 0x08, 0xf2, 0x73, 0xe6
            0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00,
            0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00,
            0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00,
            0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00, 0x00,
    };
    byte cipher[];
    static final int port = 2390;

    /* name of program, used for logging (Log.d(), etc) */
    private static final String TAG = "key";

    /* global socket, used by whole program for sending and receiving. */
    DatagramSocket socket;
    /**
     * ATTENTION: This was auto-generated to implement the App Indexing API.
     * See https://g.co/AppIndexing/AndroidStudio for more information.
     */
    private GoogleApiClient client;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_key);

        Button b_lock = (Button) findViewById(R.id.b_lock);
        Button b_unlock = (Button) findViewById(R.id.b_unlock);
        Button b_auth = (Button) findViewById(R.id.b_auth);

        /* aessend for encryption, send for unencrypted */
        b_lock.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                aessend("l");
            }
        });
        b_unlock.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                aessend("u");
            }
        });
        b_auth.setOnClickListener(new View.OnClickListener() {
```

```
            int count = bytes.length;
            try {
                //InetAddress ip = getBroadcastAddress();
                for (int i = 0; i < count; i++) {
                    byte[] data = bytes[i];
                    socket.setBroadcast(true);
                    DatagramPacket packet =
                            new DatagramPacket(data, data.length,
                                    getBroadcastAddress(), port);
                    socket.send(packet);
                }
            } catch (Exception e) {
                Log.e(TAG, "exception", e);
            }
            return null; /* func is of type Void (the object).  */
        }
    }

    DatagramPacket receive() throws IOException {
        byte[] buf = new byte[1024];
        DatagramPacket packet = new DatagramPacket(buf, buf.length);
        socket.receive(packet);
        return packet;
    }

    void aessend(String cmd) {
        new send().execute(cmd); /* auth */
//        new send().execute("a"); /* auth */
        /* get iv */
//        try {
//            //DatagramPacket pack = receive();
//            byte[] buf = new byte[1024];
//            DatagramPacket packet = new DatagramPacket(buf, buf.length);
//            socket.receive(packet);
//
//            Log.d(TAG, String.valueOf(packet));
//        } catch (Exception e) {
//            Log.e(TAG, "didn't get response from socket");
//        }
//
//        try {
//            SecretKeySpec k = new SecretKeySpec(key, "AES");
//
//            Cipher c = Cipher.getInstance("AES/CBC/PKCS5Padding");
//            IvParameterSpec ivParameterSpec = new IvParameterSpec(iv);
//            c.init(DECRYPT_MODE, k, new IvParameterSpec(iv));
//            /* hopefully 128 bits. encrypt cmd */
//            byte[] ciphertext = c.doFinal(cmd.getBytes());
//            new sendbytes().execute(ciphertext); /* TODO: encrypt */
//        } catch (NoSuchAlgorithmException e1) {
//            e1.printStackTrace();
//        } catch (NoSuchPaddingException e2) {
//            e2.printStackTrace();
//        } catch (InvalidAlgorithmParameterException e) {
//            e.printStackTrace();
//        } catch (InvalidKeyException e) {
//            e.printStackTrace();
//        } catch (BadPaddingException e) {
//            e.printStackTrace();
//        } catch (IllegalBlockSizeException e) {
```

```
            @Override
            public void onClick(View v) {
                //new send().execute("a");
                aessend("a");
            }
        });

        /* create socket now so both send and receive methods can use it later */
        try {
            socket = new DatagramSocket(port);
            socket.setBroadcast(true);
        } catch (Exception e) {
        }
        // ATTENTION: This was auto-generated to implement the App Indexing API.
        // See https://g.co/AppIndexing/AndroidStudio for more information.
        client = new GoogleApiClient.Builder(this).addApi(AppIndex.API).build();
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if it is present.
        getMenuInflater().inflate(R.menu.menu_key, menu);
        return true;
    }

    @Override
    public boolean onOptionsItemSelected(MenuItem item) {
        // Handle action bar item clicks here. The action bar will
        // automatically handle clicks on the Home/Up button, so long
        // as you specify a parent activity in AndroidManifest.xml.
        int id = item.getItemId();

        //noinspection SimplifiableIfStatement
        if (id == R.id.action_settings) {
            return true;
        }

        return super.onOptionsItemSelected(item);
    }

    /* https://code.google.com/p/boxeeremote/wiki/AndroidUDP
   used in send() */
    InetAddress getBroadcastAddress() throws IOException {
        WifiManager wifi = (WifiManager) getSystemService(Context.WIFI_SERVICE);
        DhcpInfo dhcp = wifi.getDhcpInfo();
        if (dhcp != null) openOptionsMenu(); /* so they can turn on wifi */

        int broadcast = (dhcp.ipAddress & dhcp.netmask) | ~dhcp.netmask;
        byte[] quads = new byte[4];
        for (int k = 0; k < 4; k++)
            quads[k] = (byte) ((broadcast >> k * 8) & 0xFF);
        return InetAddress.getByAddress(quads);
    }

    @Override
    public void onStart() {
        super.onStart();

        // ATTENTION: This was auto-generated to implement the App Indexing API.
        // See https://g.co/AppIndexing/AndroidStudio for more information.
```

```
        client.connect();
        Action viewAction = Action.newAction(
                Action.TYPE_VIEW, // TODO: choose an action type.
                "key Page", // TODO: Define a title for the content shown.
                // TODO: If you have web page content that matches this app
activity's content,
                // make sure this auto-generated web page URL is correct.
                // Otherwise, set the URL to null.
                Uri.parse("http://host/path"),
                // TODO: Make sure this auto-generated app deep link URI is correct.
                Uri.parse("android-app://com.cubesolving.jns.key/http/host/path")
        );
        AppIndex.AppIndexApi.start(client, viewAction);
    }

    @Override
    public void onStop() {
        super.onStop();

        // ATTENTION: This was auto-generated to implement the App Indexing API.
        // See https://g.co/AppIndexing/AndroidStudio for more information.
        Action viewAction = Action.newAction(
                Action.TYPE_VIEW, // TODO: choose an action type.
                "key Page", // TODO: Define a title for the content shown.
                // TODO: If you have web page content that matches this app
activity's content,
                // make sure this auto-generated web page URL is correct.
                // Otherwise, set the URL to null.
                Uri.parse("http://host/path"),
                // TODO: Make sure this auto-generated app deep link URI is correct.
                Uri.parse("android-app://com.cubesolving.jns.key/http/host/path")
        );
        AppIndex.AppIndexApi.end(client, viewAction);
        client.disconnect();
    }

    /* http://developer.android.com/reference/android/os/AsyncTask.html */
    private class send extends AsyncTask<String, Void, Void> {
        protected Void doInBackground(String... strings) {
            int count = strings.length;
            try {
                //InetAddress ip = getBroadcastAddress();
                for (int i = 0; i < count; i++) {
                    String data = strings[i];
                    socket.setBroadcast(true);
                    DatagramPacket packet =
                            new DatagramPacket(data.getBytes(), data.length(),
                                    getBroadcastAddress(), port);
                    socket.send(packet);
                }
            } catch (Exception e) {
                Log.e(TAG, "exception", e);
            }
            return null; /* func is of type Void (the object).  */
        }
    }

    /* http://developer.android.com/reference/android/os/AsyncTask.html */
    private class sendbytes extends AsyncTask<byte[], Void, Void> {
        protected Void doInBackground(byte[]... bytes) {
```

```
/*
 Arduino Wireless Lock by Joshua And Marcus

 Run this on an Arduino with a WiFi Shield.
 in a terminal run
nc -u [shield's ip] 2390
  e.g.
nc -u 192.168.1.129 2390
 send l to sim. lock and u to sim. unlock. a light will blink.

 based on:
 WiFi UDP Send and Receive String
 This sketch wait an UDP packet on localPort using a WiFi shield.
 When a packet is received an Acknowledge packet is sent to the client on port
remotePort

 created 30 December 2012
 by dlf (Metodo2 srl)

 thank Robert Bares http://forum.arduino.cc/index.php?topic=38107.0
 for hex printing code
*/

#include <SPI.h>
#include <WiFi.h>
#include <WiFiUdp.h>
#include <Servo.h>
#include <AES.h>

enum {
  /* degress for servo on door lock */
  LOCK = 100,
  UNLOCK = 0,

  /* servo port */
  MOTOR = 9,

  /* auth button pin */
  PAUTH = 3,

  /* aes */
  KEYBITS = 128,
  MSGBLOCKS = 1,
  IV_LEN = 16,
  BYTE_MAX = 255,

  LOCALPORT = 2390,
  CHARS_PER_BYTE = 2,
};

char packetBuffer[IV_LEN + KEYBITS]; //buffer to hold incoming packet
char  ReplyBuffer[] = "ack\n"; // a string to send back
Servo servo;
WiFiUDP Udp;
AES aes;


byte key[] =
{
//  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
```

```
//              e.printStackTrace();
//          }

        }
    }
```

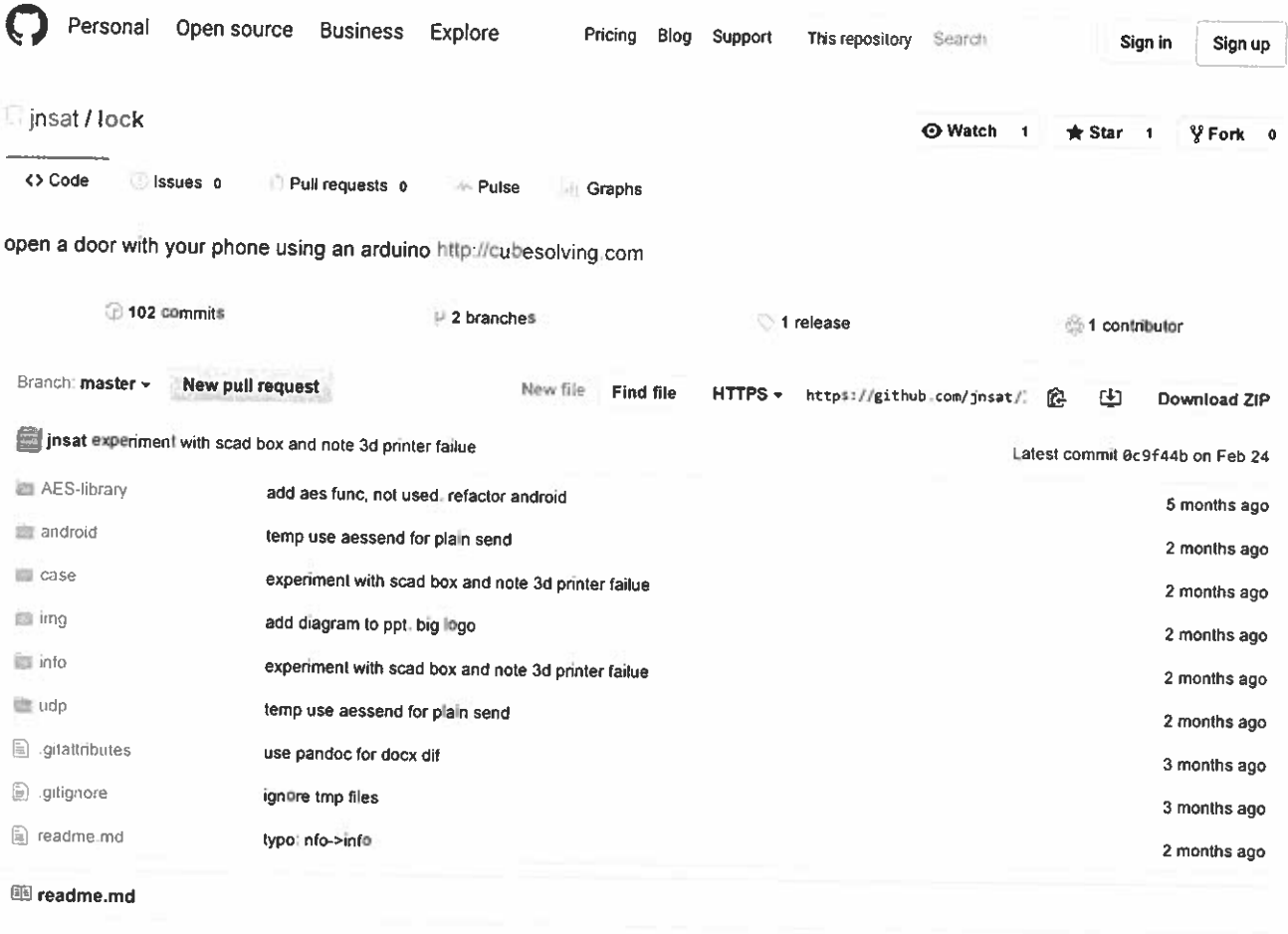
```
0x00, 0x00, 0x00,
//  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
0xa7, 0x74, 0xde, 0x71,
0xad, 0x17, 0x1d, 0xbd,
0xed, 0x3d, 0x4c, 0xc9,
0x55, 0xea, 0xb6, 0xb9,

0x1c, 0x0b, 0xed, 0x76,
0x63, 0x2d, 0x69, 0x42,
0xe7, 0xd5, 0x00, 0xd5,
0x58, 0x19, 0xbe, 0xb4,
};

byte plain[] =
{
  // 0xf3, 0x44, 0x81, 0xec, 0x3c, 0xc6, 0x27, 0xba, 0xcd, 0x5d, 0xc3, 0xfb, 0x08,
0xf2, 0x73, 0xe6
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
  0xC0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
  0xE0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
  0xF0, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x00,
};

/* randomized later */
byte my_iv[] =
{
  0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
0x00, 0x00, 0x01,
};

void p(byte X) { /* http://stackoverflow.com/questions/19127945/how-to-serial-print-
full-hexadecimal-bytes */
   if (X < 16) {Serial.print("0");}
   Serial.print(X, HEX);
   Serial.print(" ");
}

void p_udp(byte X) { /* http://stackoverflow.com/questions/19127945/how-to-serial-
print-full-hexadecimal-bytes */
   if (X < 16) {Udp.print("0");}
   Udp.print(X, HEX);
   //Udp.print(" ");
}

void
print_iv_udp(unsigned char *iv)
{
  for (int i = 0; i < IV_LEN; i++)
    p_udp(iv[i]);
}

void print_iv(unsigned char *iv) {
  for (int i = 0; i < IV_LEN; i++)
    p(iv[i]);
}
```

```
void
hex2char(byte *bytes, char *out, int len, int spaces)
{
    int char_per_byte = CHARS_PER_BYTE + spaces; /* high bit, low bit, optional
space 0x[AB ] */
    for (int i = 0; i < len; i++) { /* convert hex to char keeping zeros */
      /* Robert Bares http://forum.arduino.cc/index.php?topic=38107.0 */
      int first = (bytes[i] >> 4) & 0x0f;
      int second = bytes[i] & 0x0f;
      sprintf(&out[i * char_per_byte], spaces ? "%01X%01X " : "%01X%01X", first,
second);
    }
}

void
debug_iv()
{
      Serial.print("my_iv: [");
      print_iv(my_iv);
      Serial.print("]\n");
}

void
gen_iv(byte iv[], int len) {
  for (int i = 0; i < IV_LEN; i++)
    iv[i] = random(BYTE_MAX); /* rand byte 0x00 to 0xff */
}

byte cipher [4 * N_BLOCK];
byte check [4 * N_BLOCK];
long t0, t1; /* something for aes */
byte succ;
byte iv [N_BLOCK]; /* for interal lib */
void
setkey(int bits, int blocks)
{
  //long t0 = micros () ;
  t0 = micros();
  succ = aes.set_key (key, bits) ;
  long t1 = micros() - t0 ;
  Serial.print ("set_key ") ; Serial.print (bits) ; Serial.print (" ->") ;
Serial.print ((int) succ) ;
  Serial.print (" took ") ; Serial.print (t1) ; Serial.println ("us") ;
}

void
encrypt(int bits, int blocks)
{
  t0 = micros () ;
  if (blocks == 1)
    succ = aes.encrypt (plain, cipher) ;
  else
  {
    for (byte i = 0 ; i < 16 ; i++)
      iv[i] = my_iv[i];
    succ = aes.cbc_encrypt (plain, cipher, blocks, iv) ;
  }
  t1 = micros () - t0 ;
  Serial.print ("encrypt ") ; Serial.print ((int) succ) ;
```

```
  Serial.print (" took ") ; Serial.print (t1) ; Serial.println ("us") ;
}

void
decrypt(int bits, int blocks)
{
  t0 = micros () ;
  if (blocks == 1)
    succ = aes.decrypt (cipher, plain) ;
  else
  {
    for (byte i = 0 ; i < 16 ; i++)
      iv[i] = my_iv[i];
    succ = aes.cbc_decrypt (cipher, check, blocks, iv) ;
  }
  t1 = micros () - t0 ;
  Serial.print ("decrypt ") ; Serial.print ((int) succ) ;
  Serial.print (" took ") ; Serial.print (t1) ; Serial.println ("us") ;

  for (byte ph = 0 ; ph < (blocks == 1 ? 3 : 4) ; ph++)
  {
    for (byte i = 0 ; i < (ph < 3 ? blocks*N_BLOCK : N_BLOCK) ; i++)
    {
      byte val = ph == 0 ? plain[i] : ph == 1 ? cipher[i] : ph == 2 ? check[i] :
iv[i];
      Serial.print (val >> 4, HEX) ; Serial.print (val & 15, HEX) ; Serial.print ("
") ;
    }
    Serial.println () ;
  }
}

void
auth(char c)
{
  debug_iv();
  gen_iv(my_iv, IV_LEN); /* randomize iv */
  debug_iv();
      /* send iv like a challenge, no need to encrypt.
      should be rand and uniq ofor real world to prevent replay */
//      Udp.beginPacket(Udp.remoteIP(), Udp.remotePort());
//      /* write iv all at once, doing each byte one at a time cuts it off after 13
out of 16 */
//      int spaces = c == 'A'; /* captial commands mean format pretty (w spaces) */
//      int char_per_byte = CHARS_PER_BYTE + spaces;
//      char buf[IV_LEN * char_per_byte];
//      hex2char(my_iv, buf, IV_LEN, spaces);
//      Udp.write(buf);
  for (int i = 0; i < IV_LEN; i++)
    Udp.write(my_iv[i]);
  Udp.endPacket();
}

void setup() {
  int status = WL_IDLE_STATUS;
  char ssid[] = "Cisco25707";
  char pass[] = "password";
//  char ssid[] = "statewide";
//  char pass[] = "st88wide";
```

```
  //pinMode(PAUTH, INPUT);
  servo.attach(MOTOR);
  randomSeed(analogRead(0)); /* seed with analog pin noise */
  setkey(KEYBITS, 1); /* aes bits */

  Serial.begin(9600);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for Leonardo only
  }

  // check for the presence of the shield:
  if (WiFi.status() == WL_NO_SHIELD) {
    Serial.println(F("WiFi shield not present"));
    // don't continue:
    while (true);
  }

  String fv = WiFi.firmwareVersion();
  if (fv != "1.1.0") {
    Serial.println(F("Please upgrade the firmware"));
  }

  // attempt to connect to Wifi network:
  while (status != WL_CONNECTED) {
    Serial.print(F("Attempting to connect to SSID: "));
    Serial.println(ssid);
    // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
    status = WiFi.begin(ssid, pass);
    // wait 10 seconds for connection:
    delay(10000);
  }
  Serial.println(F("Connected to wifi"));
  printWifiStatus();

  Serial.println(F("\nStarting connection to server..."));
  // if you get a connection, report back via serial:
  //WiFi.config(IP); // not possible to set static ip for udp...
  Udp.begin(LOCALPORT);

  encrypt(128, 4);
  decrypt(128, 4);
  Serial.println(F("Setup done"));
}

void loop() {
  /* test auth setup button. could be used to send aes key accross network once. */
  //Serial.println("PAUTH");
  //Serial.println(digitalRead(PAUTH));

  // if there's data available, read a packet
  int packetSize = Udp.parsePacket();
  if (packetSize) {
    Serial.print(F("Received packet of size "));
    Serial.println(packetSize);
    Serial.print(F("From "));
    //IPAddress remoteIp = Udp.remo();
    //Serial.print(remoteIp);
    Serial.print(F(", port "));
    Serial.println(Udp.remotePort());
```

```
    // read the packet into packetBufffer
    int len = Udp.read(packetBuffer, 255);
    if (len > 0) packetBuffer[len] = 0;
    Serial.print(F("Contents: {"));
    Serial.print(packetBuffer);
    Serial.println(F("]"));
    //decrypt(packetBuffer);
    decrypt(KEYBITS, 5); /* what's bits? */
    char c = packetBuffer[0];
    plain[0] = c; /* first char */
    //decrypt(KEYBITS, MSGBLOCKS);
    if (c == 'l') { /* lock */
      servo.write(LOCK);
      Serial.println("lock");
    }
    else if (c == 'u') { /* unlock */
      servo.write(UNLOCK);
      Serial.println("unlock");
    }
    else if (c == 'a' || c == 'A') {
      auth(c);
    } else {
     Serial.printf(F("err: did not understand command:[%s]\n"), packetBuffer);
    }
    // send a reply, to the IP address and port that sent us the packet we received
    Udp.beginPacket(Udp.remoteIP(), Udp.remotePort());
    Udp.write(ReplyBuffer);
    Udp.endPacket();
    Serial.print(Udp.remoteIP());
    Serial.print(F(" "));
    Serial.print(Udp.remotePort());
  }
}

void printWifiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print(F("SSID: "));
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print(F("IP Address: "));
  Serial.println(ip);

  // print the received signal strength:
  long rssi = WiFi.RSSI();
  Serial.print(F("signal strength (RSSI):"));
  Serial.print(rssi);
  Serial.println(F(" dBm"));
}
```



lock

Unlock a door wirelessly using your phone. Uses an Android app to send commands over UDP to and Arduino.

auth

parts

- Arduino
- Arduino Wi-Fi Shield [1]
- Android phone (firmware around 23 or higher)
- Wi-Fi
- Motor (Arduino code is currently set up for 3 wire servo)

[1]
http://www.amazon.com/Arduino-WiFi-Shield/dp/B00MEKEBXG

build

To wire the Arduino connect the servo like this

Servo	Arduino
White	pin 9

Servo	Arduino
Red	5v
Black	gnd

Make an enclosure. For our first one we mounted the servo on top of the deadbolt int a metal case. Alternatively, if you want to use the second case (work in progress), which is 3d printed:

- convert case/box.scad to stl
- print using replicatorG

install

- Patch your arduino library to use printf if needed (used in current version) using this guide http://playground.arduino.cc/Main/Printf
- open udp/udp.ino in the Ardiuno IDE, change the SSID and password to match yours. upload to Ardiuno.
- Open android app inside android/ folder using Android Studio. upload to phone by clicking the play symbol (for debug) or another way.

files

```
android/    android key app
udp/    arduino lock code
info/   presentations etc.
case/   scad file for 3d printed enclosure!
```

encryption

Working on using AES CBC.

[X] Send random IV

todo

- design nice case
- 3d print the case
- status led
- multi user?
- battery

useful tools

What	Why
Android Studio	dev and run android app
Arduino IDE	flash Arduino
OpenSCAD	3d model by programming
ReplicatorG	convert or send model to 3d printer

external docs

Android Developer
Arduino WiFI
OpenSCAD cheatsheet
There was a nice Android UDP guide on Google code, I saved it in info/AndroidUDP.md

thanks

Thanks to Dr. Lin for the guidance and Marcus for the hardware

```
arduino wifi chat
=================
We want to turn a motor forwards or backwards depending on the message sent to the Arduino.
The motor has been set up already but to make testing easier I am just blinking two LEDs,
one for each direction.

I want to talk to the wifi shield from my laptop.
I want to send a message to it from a phone eventually but for know I'm using nc (netcat)
to test it.

Check out the [WiFi](https://www.arduino.cc/en/Reference/WiFi) docs. They are nice and
have some good examples. Check out the ones inside the pages for the functions and
classes too, like [WiFiServerWrite](https://www.arduino.cc/en/Reference/WiFiServerWrite).

## nc
Check out the manual (`man nc` or `man netcat`).
In a terminal this is how you can send a packet with netcat

    nc [ip] [port]
Use the ip of the arduino wifi shield an the port it is using.
For example

    nc 192.168.1.129 2390
Once it is running, type something and press enter.
It will send the text to the shield.
To listen, use `-l`

    nc -l 2390
This will listen for packets coming to your computer.
When you are connected to a device it will get messages from it even without using `-l`.
You can't use `-l` with some of the other options. See the manual:

    -l      Used to specify that nc should listen for an incoming connection
            rather than initiate a connection to a remote host.  It is an
            error to use this option in conjunction with the -p, -s, or -z
            options.  Additionally, any timeouts specified with the -w option
            are ignored.
Note the `-u` option

    -u      Use UDP instead of the default option of TCP.

My professor recommended udp so I may use this option later.

## talking
    [14:22] <Xeek> markson: looks like tcp, smells like tcp
    [1]
The part of the wifi library that use the client.xyz and server.xyz appear to use tcp.

Using an example chat program and adding blinking code we can make a quick demo to
control the light
```
```
/*
 Chat  Server

 A simple server that distributes any incoming messages to all
 connected clients.  To use telnet to  your device's IP address and type.
 You can see the client's input in the serial monitor as well.

 This example is written for a network using WPA encryption. For
 WEP or WPA, change the Wifi.begin() call accordingly.


 Circuit:
 * WiFi shield attached
```

```
 created 18 Dec 2009
 by David A. Mellis
 modified 31 May 2012
 by Tom Igoe

 */

#include <SPI.h>
#include <WiFi.h>
#define OPEN 8
#define CLOSE 9

char ssid[] = "Cisco25707";  //  your network SSID (name)
char pass[] = "password";    // your network password
int status = WL_IDLE_STATUS;

WiFiServer server(2390);

boolean alreadyConnected = false; // whether or not the client was connected previously
void
blink(int pin)
{
  digitalWrite(pin, HIGH);
  delay(1000);
  digitalWrite(pin, LOW);
  delay(1000);
}
void setup() {
    pinMode(OPEN, OUTPUT);
  pinMode(CLOSE, OUTPUT);
  //Initialize serial and wait for port to open:
  Serial.begin(9600);
  while (!Serial) {
    ; // wait for serial port to connect. Needed for Leonardo only
  }

  // check for the presence of the shield:
  if (WiFi.status() == WL_NO_SHIELD) {
    Serial.println("WiFi shield not present");
    // don't continue:
    while(true);
  }

  // attempt to connect to Wifi network:
  while ( status != WL_CONNECTED) {
    Serial.print("Attempting to connect to SSID: ");
    Serial.println(ssid);
    // Connect to WPA/WPA2 network. Change this line if using open or WEP network:
    status = WiFi.begin(ssid, pass);

    // wait 10 seconds for connection:
    delay(10000);
  }
  // start the server:
  server.begin();
  // you're connected now, so print out the status:
  printWifiStatus();
 }


void loop() {
  // wait for a new client:
  WiFiClient client = server.available();
```

```
  // when the client sends the first byte, say hello:
  if (client) {
    if (!alreadyConnected) {
      // clead out the input buffer:
      client.flush();
      Serial.println("We have a new client");
      client.println("Hello, client!");
      alreadyConnected = true;
    }

    if (client.available() > 0) {
      // read the bytes incoming from the client:
      char thisChar = client.read();
         if (thisChar == 'l') /* lock */
      blink(CLOSE);
    if (thisChar == 'u') /* unlock */
      blink(OPEN);
      // echo the bytes back to the client:
      server.write(thisChar);
      // echo the bytes to the server as well:
      Serial.write(thisChar);
    }
  }
}


void printWifiStatus() {
  // print the SSID of the network you're attached to:
  Serial.print("SSID: ");
  Serial.println(WiFi.SSID());

  // print your WiFi shield's IP address:
  IPAddress ip = WiFi.localIP();
  Serial.print("IP Address: ");
  Serial.println(ip);

  // print the received signal strength:
  long rssi = WiFi.RSSI();
  Serial.print("signal strength (RSSI):");
  Serial.print(rssi);
  Serial.println(" dBm");
}
```

Listen using nc (`nc 192.168.1.129 2930` for me).
Type `l` to blink pin 9 and `u` to blink 8.
***

[1]: #arduino on irc.freenode.net:

    [14:10] <markson> Hi guys. Does the Arduino wifi class use tcp by default? in the examples when it just says server.xyz and client.xyz like https://www.arduino.cc/en/Tutorial/WiFiChatServer
    ...
    [14:22] <Xeek> markson: looks like tcp, smells like tcp
    [14:20] <Xeek> markson: I could look
    [14:21] == sputnikchen [~Sputnik@178-191-129-44.adsl.highway.telekom.at] has joined #arduino
    [14:22] <Xeek> markson: looks like tcp, smells like tcp
    [14:22] <Xeek> taste it?
    [14:22] <deshipu> if it smells like chicken, you are holding the soldering iron wrong
    [14:23] <sputnikchen> Hello, can i use an arduino with USB to program an arduiono without USB?

[14:23] <Xeek> yeah... soldered chicken should smell like it was grilled

```
arduino wifi
============
We tried using an Xbee Zigbee.
They are for mesh communications so they don't do Wifi.
Then we tried to use and Xbee Wifi.
I couldn't get it to initialize so I got an Arduino Wifi shield.
## docs
[ArduinoWiFiShield](https://www.arduino.cc/en/Main/ArduinoWiFiShield)
[WiFiShieldFirmwareUpgrading](https://www.arduino.cc/en/Hacking/WiFiShieldFirmwareUpgrading
)
[ArduinoWiFiShield](https://www.arduino.cc/en/Guide/ArduinoWiFiShield)
[WiFi](https://www.arduino.cc/en/Reference/WiFi)

## firmware
arduino.cc advises upgrading the firmware first. Here is a summary:
To upgrade the firmware first download a programmer.
* Windows: [Atmel's flip](http://www.atmel.com/tools/FLIP.aspx)
* Mac: install dfu-programmer using
[macports](https://www.macports.org/install.php#pkg).
  In Yosemite the installer stopped on the pentultimate step. Just quit it [1] (don't
  sue). Run:
      sudo port selfupdate
      sudo port install dfu-programmer
      sudo port upgrade outdated
* Other: install `dfu-programmer`

There are firmware installation scripts in
`/Applications/Arduino.app/Contents/Java/hardware/arduino/avr/firmwares/wifishield/scripts`

You might save a copy of the right script somewhere easy to get to. Use the one ending in
'_mac' if you're on a mac.

Connect the J3 jumper. It is a small rectangular piece of plastic which comes attached to
one small metal pole on the shield. Pull it up and put it over both of the pins there.
Then run:

    chmod a+x ./ArduinoWifiShield_upgrade_mac.sh
    ./ArduinoWifiShield_upgrade_mac.sh -a /Applications/Arduino.app/Contents/Java -f shield


## code
Example (based on arduino.cc's)
```
/* code from https://www.arduino.cc/en/Guide/ArduinoWiFiShield
 * https://www.arduino.cc/en/Reference/WiFiLocalIP
 * etc. public domain according to them.
 */
#include <SPI.h>
#include <WiFi.h>

char ssid[] = "ssid";
char pass[] = "password";
int status = WL_IDLE_STATUS; // wifi radio's status
IPAddress ip;

void setup() {1
  Serial.begin(9600);
  Serial.println("Attempting to connect to WPA network...");
  status = WiFi.begin(ssid, pass);

  if (status != WL_CONNECTED) {
    Serial.println("Couldn't get a wifi connection");
    while(true);
  }
```
```

```
  else {
    Serial.println("Connected to network");
    ip = WiFi.localIP();
    Serial.println(ip);
  }
}

void loop() {
}
```

[1]: I tried just closing it because of https://trac.macports.org/ticket/43228

Challenges:
After trying to pad sprintf for a while I discover
http://forum.arduino.cc/index.php?topic=179159.0
"The variable width or precision field (an asterisk * symbol) is not realized and will to abort the output."
Great. I went on to use sprintf with a buffer, see code.

How to diff word docs
http://blog.martinfenner.org/2014/08/25/using-microsoft-word-with-git/

```
# .gitattributes file in root folder of your git project
*.docx diff=pandoc

# .gitconfig file in your home folder
[diff "pandoc"]
  textconv=pandoc --to=markdown
  prompt = false
[alias]
  wdiff = diff --word-diff=color --unified=1
```

You can then use git wdiff important_file.docx to see the changes (with deletions in red and insertions in green), or git log -p --word-diff=color important_file.docx to see all changes over time.

Wed Feb 24
Recently I tried to print a case with a 3d printer. It's a fFlashForge. The first lid I printed was small and fragile. Up tried making a thicker one but I did not watch it and I giant glob of plastic built up on the extruder and the printer broke. It won't turn on. If you move turn a motor quickly, it acts as a generator and powers the LCD backlight. We sawed some of the plastic off. The power supply works, its LED is on. The power in on the motherboard looks burnt, and so does a chip that says
  AMS1117
  3.3 1308
Yikes.
The good thing is I looked that up and it's a voltage regulator, so maybe it blew up and saved the rest of the board.
There is an overheat led that reportedly ligths up if you have voltage. It's off.

I tired hooking up the Arduino to a battery. You can plug + to VIN and - to GND[1]. Two AA's doesn't seem to be enough -- the led lights up half way and I can't controll it with WiFi.
[1] see comments:
http://letsmakerobots.com/content/add-9v-battery-snap-arduino-without-jack